

4.9 Order of evaluation

precedence rules

The order in which operators are evaluated in an expression is known as the order of evaluation.

Table 4.9.1: Precedence rules for arithmetic, logical, and relational operators.

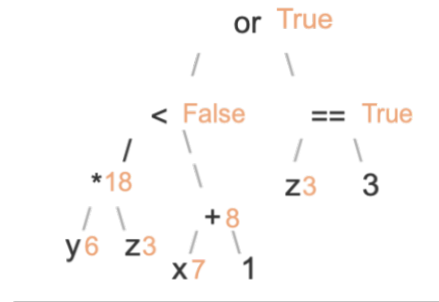
Operator/Convention	Description	Explanation
()	Items within parentheses are evaluated first	In <code>(a * (b + c)) - d</code> , the <code>+</code> is evaluated first.
<code>* / % + -</code>	Arithmetic operators (using their precedence rules; see earlier section)	<code>z = 45 * y < 53</code> evaluates <code>*</code> first.
<code>< <= > >= == !=</code>	Relational, (in)equality, and membership operators	<code>x < 2 or x >= 10</code> is evaluated as <code>x < 2</code> because <code><</code> and <code>>=</code> have precedence over <code>or</code> .
<code>not</code>	not (logical NOT)	<code>not x or y</code> is evaluated as <code>(not x) or y</code> .
<code>and</code>	Logical AND	<code>x == 5 or y == 10 and z != 10</code> is evaluated as <code>(x == 5) or ((y == 10) and z != 10)</code> because <code>and</code> has precedence over <code>or</code> .
<code>or</code>	Logical OR	<code>x == 7 or x < 2</code> is evaluated as <code>x == 7</code> because <code><</code> and <code>==</code> have precedence over <code>or</code> .

**PARTICIPATION
ACTIVITY**

4.9.1: Applying the precedence rules to an expression can be thought of as a tree.

Start ☐ 2x speed

$y * z < x + 1 \text{ or } z == 3$



Captions ^

1. Expressions like $y * z < x + 1 \text{ or } z == 3$ are evaluated using precedence rules. Arithmetic o
2. Relational operators like $<$ and $==$ come next, and the logical operator or comes last.
3. The expression is actually treated like a tree, evaluated from the bottom up.
4. If x is 7, y is 6, and z is 3, then $y * z$ is 18. Next, $x + 1$ is 8. Next, $18 < 8$ is False. Next, $z == 3$ is True.

CHALLENGE ACTIVITY

4.9.1: Order of evaluation.

763650.1722066.qx3zqy7

Start

Given the following expression, determine the order that the operator is evaluated in.

$c \text{ or } d \text{ and } f$

1)

2)

1	2	3	4
---	---	---	---

Check

Next

View solution ▼ *(Instructors only)*